D.2 Field types

Table D.2 contains the fields that ship with the Django ORM. All fields support common field arguments described in D.1. Field-specific arguments are highlighted in the field's description in the table.

Some fields, like `EmailField` and `URLField`, automatically use a field validator. For a complete list of validators, see https://docs.djangoproject.com/en/dev/ref/validators/.

Table D.2 ORM field types

Field	Description
`AutoField(**options)`	An `IntegerField` that automaticaly increments. This is not typically used explicitly, as Django includes an ID field using this option automatically.
`BigAutoField(**options)`	This is a 64-bit version of `AutoField`; newer versions of Django use this as the ID field by default.
`BinaryField(max_length=None, **options)`	This stores raw binary data. An optional `max_length` argument specifies storage size in the database.
`CharField(max_length=None, **options)`	A string field, the `max_length` option maps to the VARCHAR column size in some databases and may be required. It uses a `TextInput` widget in forms by default (rendered as `<input>`).
`DateField(auto_now=False, auth_now_add=False, **options)`	This stores a Python `datetime.date` instance. If `auto_now` is `True`, the field is set to the current date each time a Model is saved. If `auto_now_add` is `True`, the field is set to the date the Model is created.
`DateTimeField(auto_now=False, auth_now_add=False, **options)`	This is similar to `DateField`, but it stores a Python `datetime.datetime` object.
`DecimalField(max_digits=None, decimal_places=None, **options)`	This stores a fixed precision decimal number represented by a Python `Decimal` object. Both `max_digits` and `decimal_places` are required arguments, with the first representing the number of digits in the number (including the decimal points) and the second representing the number of decimal digits.
`DurationField(**options)`	This stores a Python `timedelta` object to capture periods of time.
`EmailField(max_length=254, **options)`	This is a `CharField` instance that validates the contents against an `EmailValidator`.